

# 清华大学《机器人与仿生学》课程大作业实验指导书

## 基于视觉伺服的机械臂抓取仿真

### 一、任务:

搭建一个 Gazebo 仿真环境，其中平面上有 0-9 的方块和 1-6 的格子，旁边固定一个六关节机器人，机械手末端安装 USB 相机和雷达，如图 1 所示。

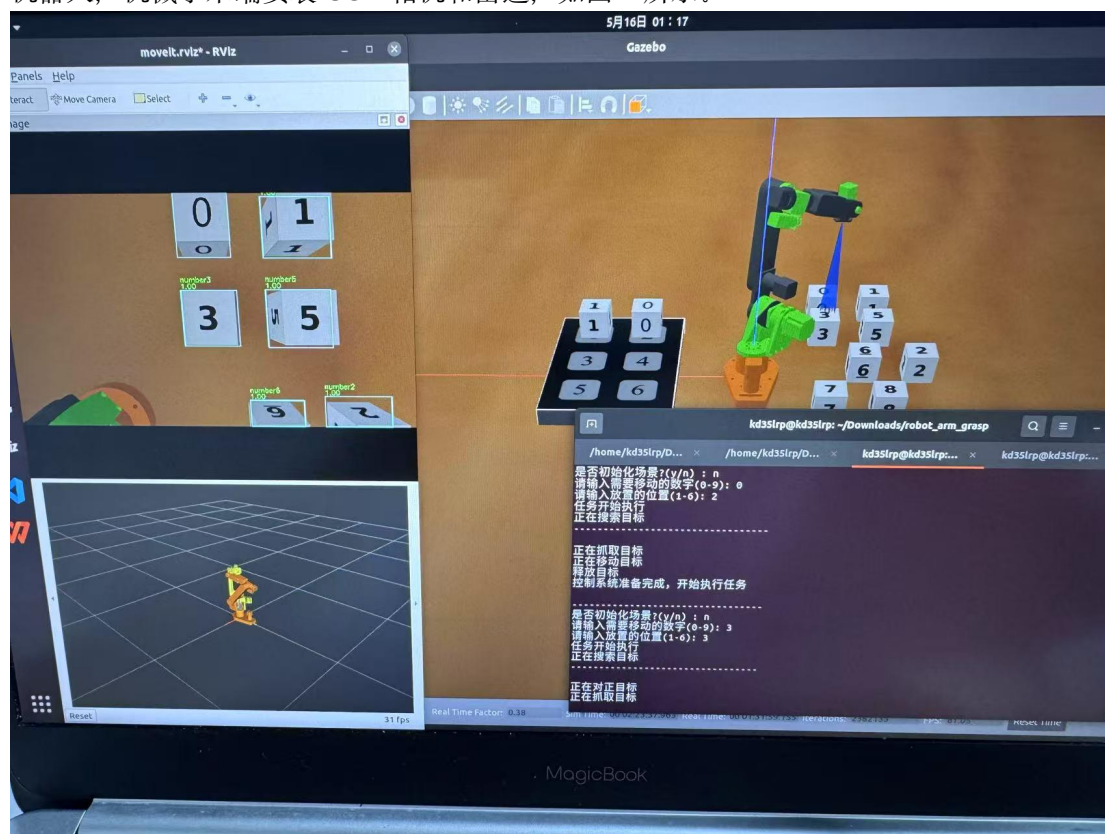


图 1 系统仿真示意图

相机实时拍照并对图像进行处理，检测到物体之后发送物体的坐标，将图像处理过程显示出来，然后规划机械臂运动路径，控制机械臂移动到物体附近进行抓取，移动到对应数字格子中。

#### 最终答辩产出:

1. 演示基础功能（可选：拓展功能）视频：需要将“0”号方块放在 1 位置，“3”号方块放在 3 位置，“1”方块放在 2 位置。
2. 整个大作业项目的代码结构和细节的说明，主要是 ROS 代码结构、ROS 各种文件类型的作用、运动功能模块、视觉功能模块等说明。
3. 最后一节课使用 PPT 答辩。

### 二、要求

#### 1. 基本功能

- (1) 在 **detection.py** 程序第 33 行和 55 行增加物体识别（使用 Yolo）和定位的功能，并

将识别到的物体在 `rivz` 中标记出来。

(2) 在 `controller.cpp` 程序第 656 行中增加机械臂对齐控制逻辑(注意没对正要反复挪动末端直到对正)和抓取功能, 学习 `moveit::planning_interface` 等 `MoveIt!` (ROS 中最流行的机器人运动规划框架) 接口。

## 2. 拓展功能

- (1) 控制机械臂末端抓取物块并实现物块的堆叠;
- (2) 添加其他物品进行识别和抓取放进数字棋盘里, 自由发挥;
- (3) 使用除了 `yolo` 以外的识别方法尝试对目标进行识别和抓取, 如传统方法或图像分割;
- (4) 尝试使用大语言模型作为 `Agent` 实现语音操控抓取指定目标物。

## 三、说明

1. 计算机操作系统: `Ubuntu20.04` (其他版本暂时缺少第三方库支持), 机器人操作系统为 `Noetic` 版本;
2. 大作业获取地址链接:  
链接: <https://pan.baidu.com/s/1ISXGRhgDrDF2EAI5NubQtw?pwd=jvpk> 提取码: `jvpk`
3. 完成基本功能只需要修改 `detection.py` 和 `controller.cpp` 两个文件中的代码添加部分即可, 完成发挥功能需要在基本功能代码基础上修改其他地方;
4. 基础代码使用可以参考下文:

# 大作业运行指南

## 大作业运行环境

`ubuntu20.04`

`ros-noetic`

由于虚拟机性能较差可能带来较长的图像识别推理时间和通信延迟, 导致任务无法执行, 需调节 `controller.cpp` 中的

```
arm.setMaxAccelerationScalingFactor(0.5)
```

```
arm.setMaxVelocityScalingFactor(0.5);
```

把速度从 0.5 调到 0.1 或者更小, 实测有效

## 配置环境

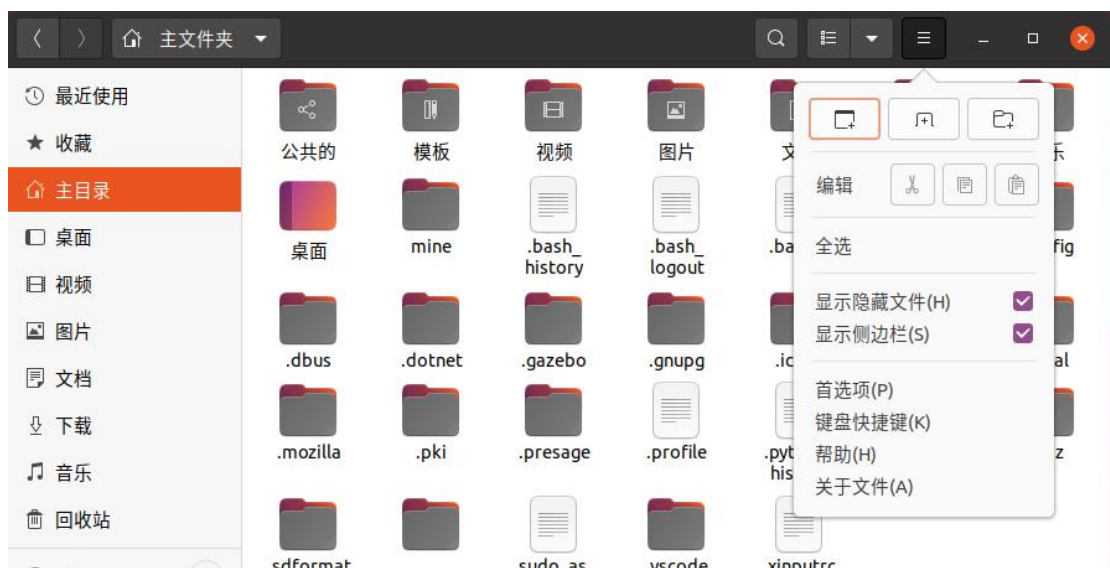
进入项目文件夹，在文件夹下打开终端，依次输入以下 3 条语句，确保无报错：

```
1.sudo apt install ros-noetic-moveit ros-noetic-joint-trajectory-controller  
ros-noetic-moveit-core ros-noetic-moveit-ros-planning  
ros-noetic-moveit-ros-planning-interface
```

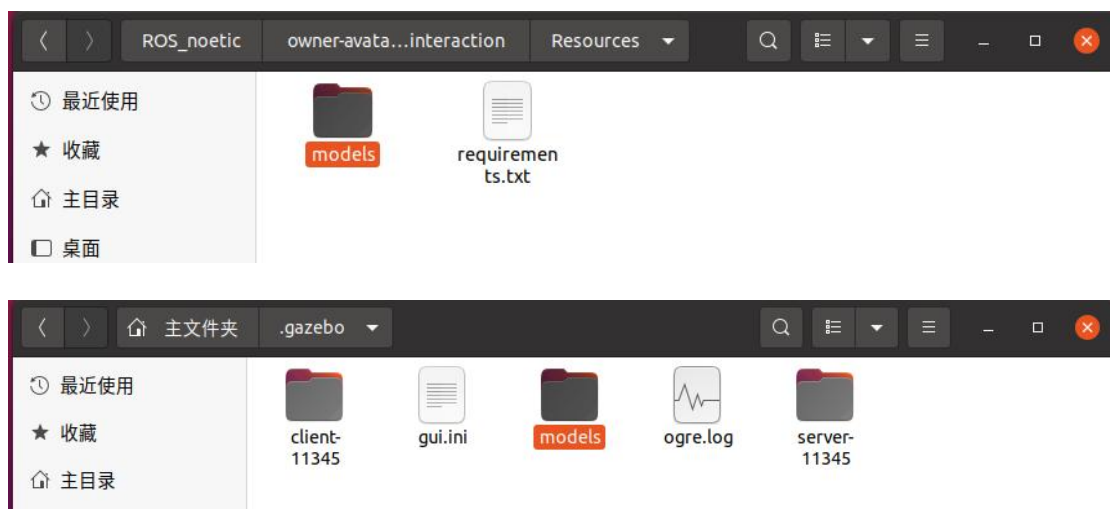
```
2.pip install -r Resources/requirements.txt
```

```
3.catkin_make
```

打开显示隐藏文件夹的开关：



进入将 Resources 文件夹中的 models 文件夹复制到主文件下的.gazebo 文件夹，进行合并



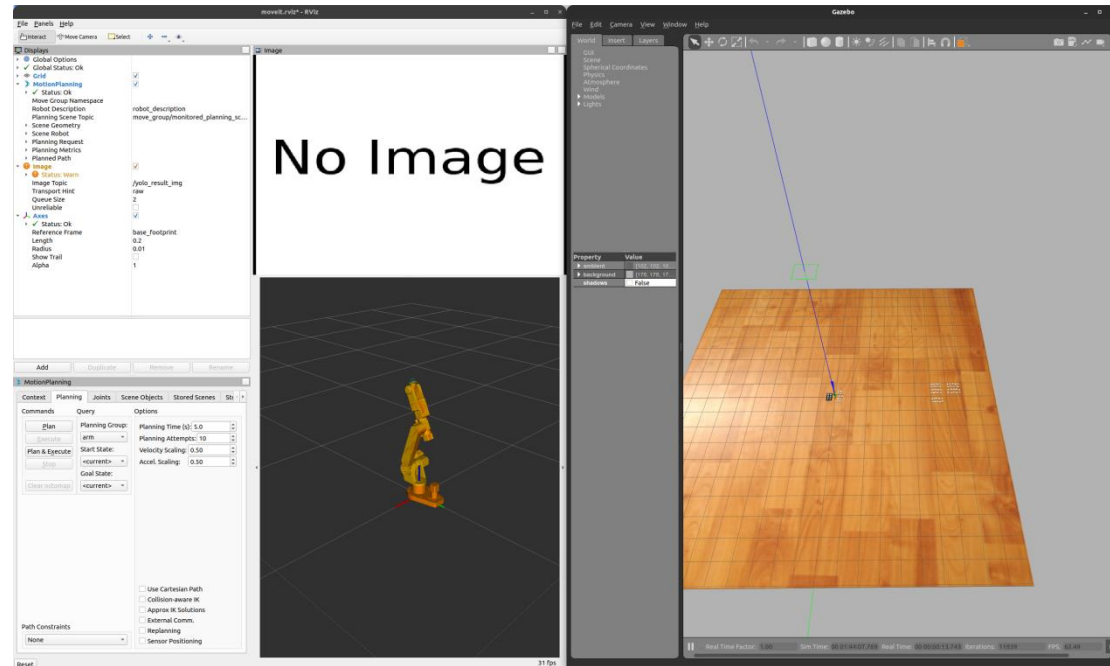
## 运行程序

在项目文件夹下打开第一个终端，依次运行：

```
source devel/setup.bash
```

```
roslaunch ar3_control moveit_gazebo.launch
```

运行成功后会启动 Gazebo 和 rviz:

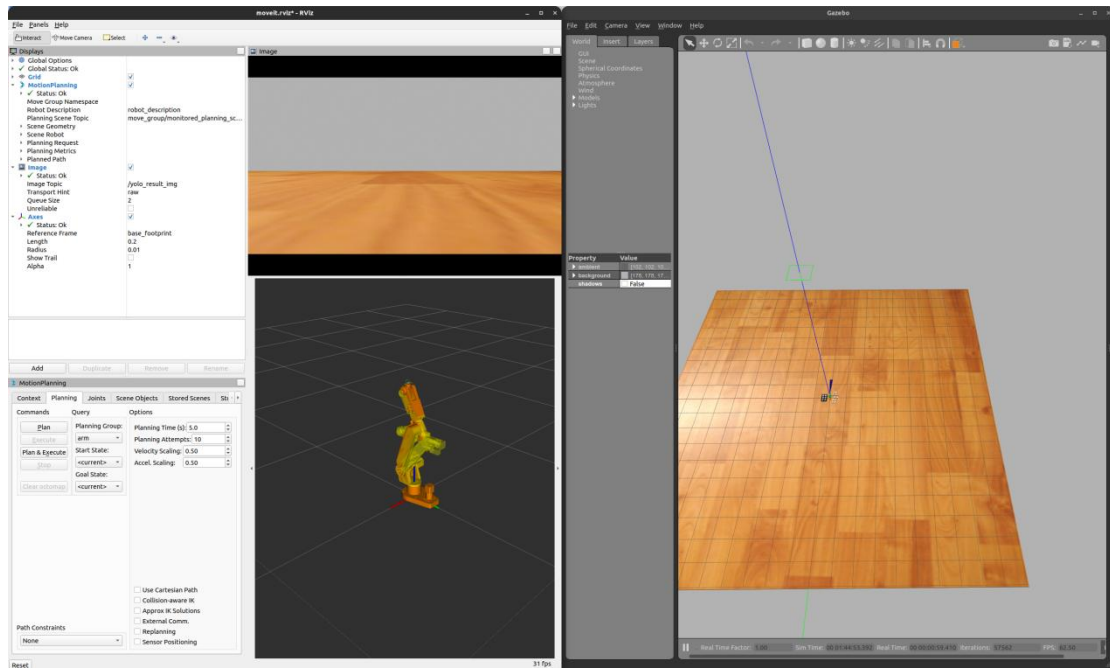


然后再在项目文件夹下打开另外一个终端，依次输入：

```
source devel/setup.bash
```

```
roslaunch ar3_control ar3_yolo_controller.launch
```

运行后 rviz 中摄像机图像将会出现，机械臂会回到初始位置



在项目文件夹下再打开另外一个终端，依次输入：

```
source devel/setup.bash
roslaunch ar3_control get_input
```

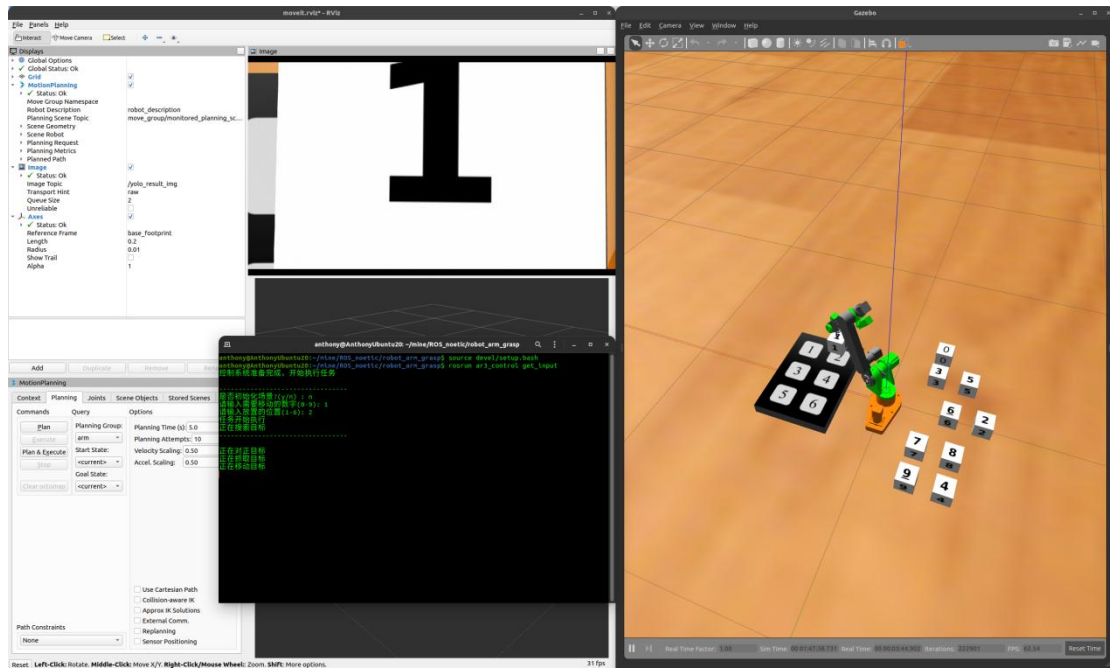
运行后会输出内容：

```
anthony@AnthonyUbuntu20:~/mine/ROS_noetic/robot_arm_grasp$ source devel/setup.bash
anthony@AnthonyUbuntu20:~/mine/ROS_noetic/robot_arm_grasp$ roslaunch ar3_control get_input
控制系统准备完成，开始执行任务

-----
是否初始化场景?(y/n) : |
```

之后在，当前终端中进行交互，机械臂会按照命令执行动作：





选择初始化场景，模型会回到初始位置:

